

Zain Asher

Software Engineer · Production AI Systems

+92 325 0288702 · zainasher42@gmail.com

linkedin.com/in/zain-asher-3960321b2 · github.com/zainasher42 · zainasher-dev.vercel.app

SUMMARY

Software engineer building production AI systems — multi-agent LLM platforms (LangGraph, 76-agent orchestration), agentic RAG with hybrid retrieval, full-stack delivery on FastAPI and Next.js. Daily user of Cursor and Claude Code. Author of github.com/zainasher42/fastapi-docs-rag — a public production-pattern RAG demo demonstrating the patterns I use in client work.

EXPERIENCE

Ember AI — Software Engineer

Sept 2025 – Present

- Architected multi-agent litigation platform orchestrating 76 specialized LLM agents (LangGraph) as a seven-wave DAG — a full regeneration is 56 steps, with waves as hard barriers and a 25-way fan-out at the widest wave. In production with paying users.
- Cost engineering (Anthropic prompt caching via stable-prefix prompt assembly, case-dossier memoization, async-bounded retrieval) that cut token spend by roughly 60% on the multi-agent system.
- Ran production retrieval on Pinecone, then led the migration to pgvector co-located in the primary Postgres — preserving the service interface so 32+ call sites were untouched.
- Built agentic RAG system for legal document analysis with hybrid retrieval (LLM-driven query generation plus curated seed queries), vision-based OCR via Gemini, ~20 single-responsibility services.
- Shipped multi-LLM media platform (artie.co) end-to-end — async FastAPI backend, Celery/Redis workers, React 18/Vite SPA, OpenAI/Gemini provider failover, deployed on AWS Amplify. Live with real users.
- Daily use of Cursor and Claude Code for AI-driven development — markdown specs as the unit of work, AI tools for implementation under engineer direction.

Devsinc — Associate Software Engineer

June 2025 – March 2026

- Built async web applications with FastAPI, MongoDB/PostgreSQL, and Redis-backed task queues; contributed to API design, Pydantic v2 schemas, and end-to-end feature delivery.

Conovo Technologies — Python Developer Intern

June 2024 – Sept 2024

- Developed web applications with Flask and Django; worked across REST APIs, ORM models, and templated UIs.

SELECTED PROJECTS

Multi-Agent Litigation Platform *Ember AI client · under NDA*

FastAPI · LangGraph · Claude (prompt caching) · Gemini · OpenAI · Pinecone → PostgreSQL + pgvector · Bedrock Titan Embed V2 · SQLAlchemy 2.0 async · Celery/Redis · ECS Fargate · Next.js

- Ingests case documents and produces motions, discovery plans, deposition outlines, expert reports, and trial themes. 76 specialized LLM agents compiled to LangGraph StateGraphs over typed state, composed into a seven-wave DAG (56 steps per full run, waves as hard barriers).
- 47 of the 76 agents retrieve; 46 run a bounded agentic loop — the model authors its own queries, then a validate node judges whether coverage is sufficient and can re-enter retrieval with new ones, capped by max_loops so it terminates.
- Ran retrieval on Pinecone (llama-text-embed-v2 embeddings, semaphore-bounded concurrency, 40 KB metadata-cap handling), then led the migration to pgvector co-located in the primary Postgres — preserving the service interface so 32+ call sites were untouched. Semantic hits now join relational facts with no second consistency model; Bedrock Titan Embed V2 pinned to 1024 dims.
- Multi-LLM provider abstraction with cost optimizations: Anthropic prompt caching (1-hour TTL), case-dossier memoization, per-provider concurrency limits, cooperative cancellation tokens giving sub-second abort. ~60% token spend reduction versus naive first version.

Legal Document Analysis Platform — Agentic RAG *Ember AI client · under NDA*

FastAPI · Celery · Pinecone · Anthropic Claude · Google Gemini · MongoDB · GCP Storage · Next.js

- Automated extraction and analysis platform for venture-financing deal documents, with async FastAPI and Celery/Redis worker pipelines.
- Multi-LLM pipeline: Gemini for vision-based OCR and text extraction, Claude for legal reasoning and classification. Agentic RAG on Pinecone with hybrid retrieval (LLM-driven query generation plus curated seed queries) and per-question routing between full-text and RAG context.
- Service-oriented backend (~20 single-responsibility services) for PDF chunking, exhibit splitting, cross-document analysis, and a legal inference engine; export to Excel and PDF.

Ask FastAPI Docs — Production-pattern RAG Demo github.com/zainasher42/fastapi-docs-rag

Python · FastAPI · async SQLAlchemy 2.0 · PostgreSQL + pgvector · Next.js · OpenAI

- Public RAG chatbot over the official FastAPI documentation. Demonstrates the patterns I use in production: hybrid retrieval (vector + keyword via Reciprocal Rank Fusion), structure-aware chunking with header-prefixed self-describing chunks, XML-structured prompts designed for caching.
- SSE streaming with structured event bus (status → citations → token × N → cost → done) for distinct retrieval and generation phases. Paired eval suite (18 hand-curated Q&A pairs, LLM-as-judge runner, 94% pass rate with honest failure reporting).

Artie — AI-Powered Brand & Campaign Platform artie.co (live)

FastAPI · MongoDB (Motor) · Celery/Redis · OpenAI + Gemini · React 18 · TypeScript · Vite · AWS Amplify · S3

- Full-stack AI media-planning and brand-matching platform with async FastAPI backend, multi-LLM layer (OpenAI/Gemini with automatic fallback), SSE streaming for real-time output, and React 18 SPA deployed via AWS Amplify.
- Celery/Redis workers for brand-project matching, the proprietary Artie Score algorithm, campaign tactics engine, YouTube-data-driven reach calculation, and pitch-deck/script generation. JWT + Argon2 + Google OAuth auth.

SKILLS

- **Languages:** Python, TypeScript, JavaScript, SQL
- **Backend:** FastAPI (async), Flask, Django, Celery, Pydantic v2, SQLAlchemy 2.0 (async), Alembic, Uvicorn, asyncpg
- **Frontend:** React 18/19, Next.js, Vite, TypeScript, TailwindCSS, shadcn/ui, TanStack Query, Zustand
- **AI / LLM:** Anthropic Claude (prompt caching), OpenAI, Google Gemini, multi-LLM orchestration with fallback, LangGraph, Agentic RAG, Pinecone (llama-text-embed-v2), pgvector, AWS Bedrock (Titan Embed V2), vector store migration, prompt engineering with XML-structured prompts
- **AI-driven Development:** Cursor, Claude Code, markdown-spec-first workflows, eval-paired prompt iteration
- **Databases & Storage:** PostgreSQL (+ pgvector), MongoDB (Motor), Redis, Pinecone; AWS S3, GCP Storage, MinIO
- **Infra & Tooling:** AWS (ECS Fargate, Amplify, S3, RDS, ElastiCache), Terraform, Docker, container deploys, SSE, JWT/Argon2/OAuth, Sentry, structlog, pytest + pytest-asyncio + httpx, document processing (pdfplumber, python-docx, python-pptx, openpyxl, reportlab)

EDUCATION

BS Computer Science — National University of Computer and Emerging Sciences, Lahore

2021 – 2025

- Relevant coursework: Data Structures, Algorithms, Database Systems, Artificial Intelligence, Data Science, Natural Language Processing, Bioinformatics.

LEADERSHIP

- Assistant Vice President, ACM NUCES Lahore Chapter (2023–2024)